Rate Monotonics

```
#include <stdio.h>
#include <math.h>   // <-- FIX: include this

#define MAX 10

int gcd(int a, int b) {
    while (b != 0) {
        int temp = b;
        b = a % b;
        a = temp;
    }
    return a;
}

int lcm(int a, int b) {
    return (a * b) / gcd(a, b);
}

int find_lcm(int arr[], int n) {
    int result = arr[0];
    for (int i = 1; i < n; i++) {
        result = lcm(result, arr[i]);
    }
    return result;
}

int main() {
    int n, i, j;
    int burst[MAX], period[MAX];
```

```
int remaining[MAX];
int hyperperiod;
float utilization = 0.0, bound;
printf("USN:1WA24CS198");
printf("\nEnter the number of processes:");
scanf("%d", &n);

printf("Enter the CPU burst times:\n");
for (i = 0; i < n; i++) {
   scanf("%d", &burst[i]);
   remaining[i] = burst[i];
}

printf("Enter the time periods:\n");
for (i = 0; i < n; i++) {
   scanf("%d", &period[i]);
}

hyperperiod = find_lcm(period, n);
printf("LCM=%d\n\n", hyperperiod);

// Sort by period (priority)
for (i = 0; i < n - 1; i++) {
   for (j = i + 1; j < n; j++) {
      if (period[i] > period[j]) {
         int temp;
         temp = period[i]; period[i] = period[j]; period[j] = temp;
         temp = burst[i]; burst[i] = burst[j]; burst[j] = temp;
         temp = remaining[i]; remaining[i] = remaining[j]; remaining[j] = temp;
      }
   }
```

```
}

printf("Rate Monotone Scheduling:\n");
printf("PID\tBurst\tPeriod\n");
for (i = 0; i < n; i++) {
   printf("%d\t%d\t%d\n", i + 1, burst[i], period[i]);
   utilization += (float)burst[i] / period[i];
}

// FIX: pow() now works
bound = n * (pow(2.0, 1.0 / n) - 1);

printf("\n%f <= %f => %s\n", utilization, bound,
      (utilization <= bound) ? "true" : "false");

printf("Scheduling occurs for %d ms\n\n", hyperperiod);

int time, current_task;

for (time = 0; time < hyperperiod; time++) {

   // Release tasks
   for (i = 0; i < n; i++) {
      if (time % period[i] == 0) {
         remaining[i] = burst[i];
      }
   }

   current_task = -1;

   // Pick highest priority task
```

```
        for (i = 0; i < n; i++) {
            if (remaining[i] > 0) {
                current_task = i;
                break;
            }
        }

        if (current_task != -1) {
            printf("%dms onwards: Process %d running\n", time, current_task + 1);
            remaining[current_task]--;
        } else {
            printf("%dms onwards: CPU is idle\n", time);
        }
    }

    return 0;
}
```

```
USN:1WA24CS198
Enter the number of processes:2
Enter the CPU burst times:
20
35
Enter the time periods:
50
100
LCM=100

Rate Monotone Scheduling:
PID     Burst   Period
1       20      50
2       35      100

0.750000 <= 0.828427 => true
Scheduling occurs for 100 ms

0ms onwards: Process 1 running
1ms onwards: Process 1 running
2ms onwards: Process 1 running
3ms onwards: Process 1 running
4ms onwards: Process 1 running
```

EDF Scheduling

```
#include <stdio.h>
#define MAX 10
int main() {
   int n;
   int burst[MAX], deadline[MAX], period[MAX];
   int remaining[MAX];
   int time = 0;
   printf("USN:1WA24CS198");
   printf("\nEnter the number of processes:");
   scanf("%d", &n);
   for (int i = 0; i < n; i++) {
      printf("Task %d (burst deadline period): ", i + 1);
      scanf("%d %d %d", &burst[i], &deadline[i], &period[i]);
      remaining[i] = 0;  // initially no job released
   }
   printf("\nPID\tBurst\tDeadline\tPeriod\n");
   for (int i = 0; i < n; i++) {
      printf("%d\t%d\t%d\t\t%d\n", i + 1, burst[i], deadline[i], period[i]);
   }
   int limit = 20;
   printf("\nScheduling occurs for %d ms\n\n", limit);

   while (time < limit) {
      for (int i = 0; i < n; i++) {
         if (time % period[i] == 0) {
            remaining[i] = burst[i];
         }
      }
      int min_deadline = 9999;
```

```
        int selected = -1;
        for (int i = 0; i < n; i++) {
            if (remaining[i] > 0) {
                if (deadline[i] < min_deadline) {
                    min_deadline = deadline[i];
                    selected = i;
                }
            }
        }
        if (selected == -1) {
            printf("%dms : CPU is idle.\n", time);
        } else {
            printf("%dms : Task %d is running.\n", time, selected + 1);
            remaining[selected]--;
        }
        time++;
    }
    return 0;
}
```

```
USN:1WA24CS198
Enter the number of processes:3
Task 1 (burst deadline period): 2 4 5
Task 2 (burst deadline period): 3 7 20
Task 3 (burst deadline period): 2 8 10

PID     Burst   Deadline        Period
1       2       4               5
2       3       7               20
3       2       8               10

Scheduling occurs for 20 ms

0ms : Task 1 is running.
1ms : Task 1 is running.
2ms : Task 2 is running.
3ms : Task 2 is running.
4ms : Task 2 is running.
5ms : Task 1 is running.
6ms : Task 1 is running.
7ms : Task 3 is running.
8ms : Task 3 is running.
```